

Cross-Platform Mobile App Development

Week 8 — Flutter (Part 2): State Management, Routing & Platform Channels

Instructor: Nguyen Thanh Tuan, PhD

Institution: Faculty of Computer Science, VKU

Today's Agenda

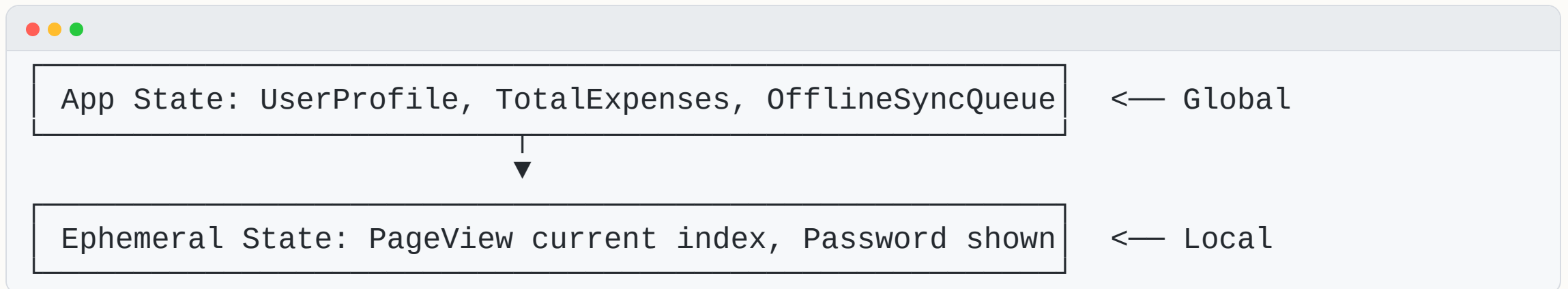
1. **State Management Spectrum** — Ephemeral vs. App State, Prop Drilling, `InheritedWidget`
2. **Provider: The Community Standard** — `ChangeNotifier` , `Consumer` , Scoped Subtree Rebuilds
3. **Riverpod 2: Modern Compile-Safe State** — `Notifier` , `ConsumerWidget` , `ref.watch` , Comparison Matrix
4. **Declarative Routing with GoRouter** — Deep Linking, Path/Query Params, `ShellRoute` Tabs, Route Guards
5. **Form Validation & Controllers** — `Form` , `GlobalKey<FormState>` , `TextEditingController` , `FocusNode`
6. **Custom Canvas Drawing (`CustomPainter`)** — Coordinate Geometry, Arcs, Animated Donut & Bar Charts
7. **Platform Channels (MethodChannel)** — Native Interop with Android (Kotlin) & iOS (Swift)
8. **Mini-Project 3 Final Sprint** — OCR Integration, Regex Heuristics, Deliverables Checklist

Part 1: The State Management Spectrum

Ephemeral State vs. App State

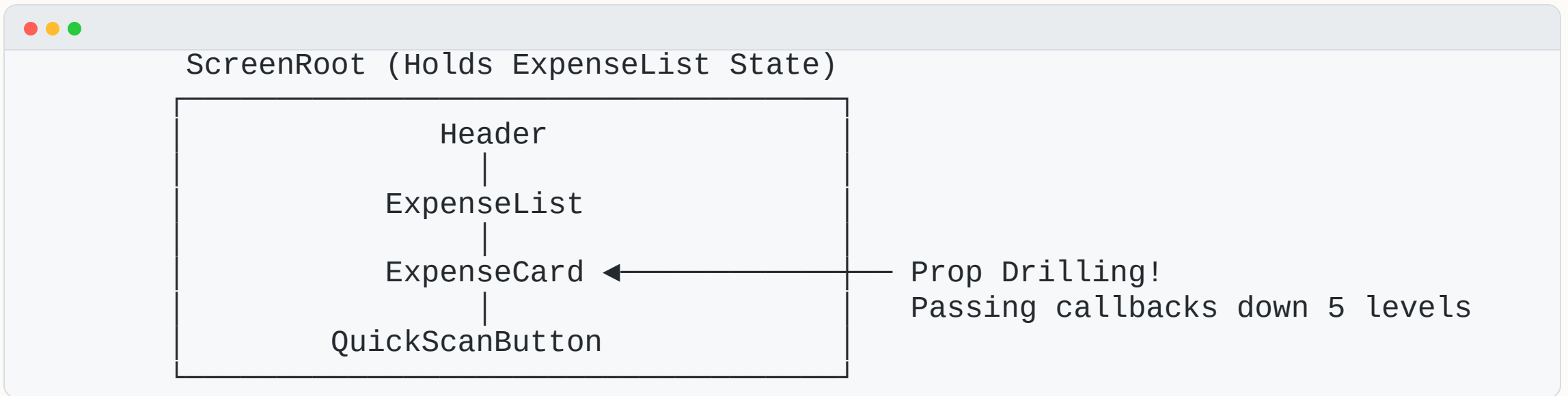
Understanding where state should live is the foundation of Flutter app architecture:

Aspect	Ephemeral (Local) State	App (Global / Shared) State
Definition	State contained neatly inside a single widget	State shared across multiple screens or services
Examples	Text field focus, tab selection, checkbox toggle, animation	Logged-in user auth, expense ledger, shopping cart, theme
Tool	<code>StatefulWidget</code> + <code>setState()</code>	Provider , Riverpod 2 , or BLoC
Lifespan	Discarded when widget unmounts	Survives screen navigation transitions



Why `setState()` Fails at Scale

While `setState()` is perfect for isolated toggles, it creates major architectural bottlenecks in real apps:



- **Prop Drilling:** Passing data and update callbacks through intermediate widgets.
- **Massive Rebuilds:** Calling `setState()` at the root reconstructs the entire page hierarchy.
- **Coupled Business Logic:** Mixes data fetching and database queries into UI presentation.

The Underlying Foundation: `InheritedWidget`

`InheritedWidget` is Flutter's built-in mechanism for propagating data down the element tree:

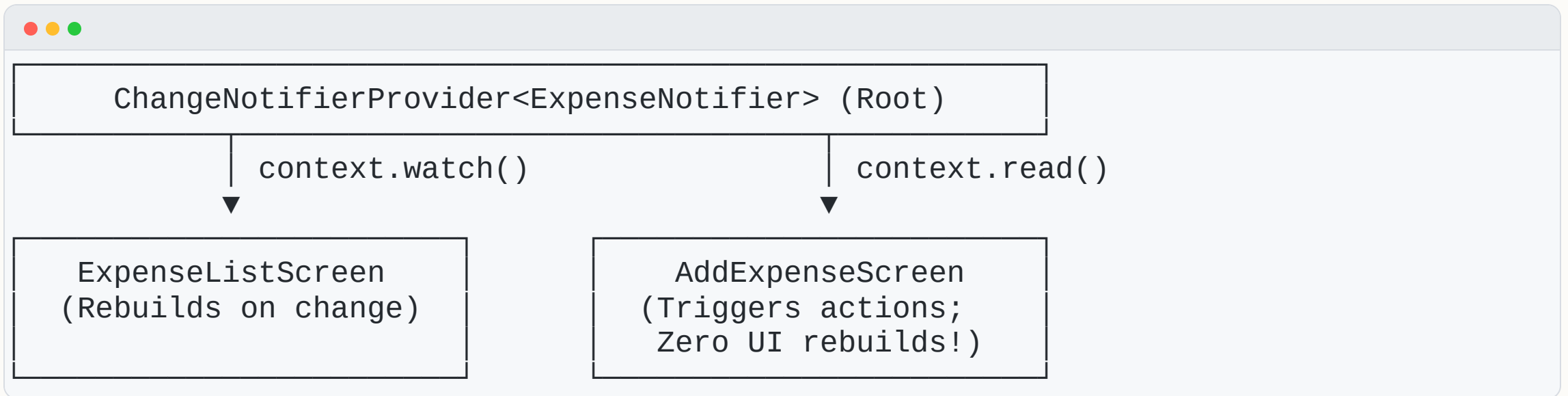
```
class ExpenseScope extends InheritedWidget {  
  final List<ExpenseItem> expenses;  
  const ExpenseScope({super.key, required this.expenses, required super.child});  
  
  static ExpenseScope? of(BuildContext context) {  
    return context.dependOnInheritedWidgetOfExactType<ExpenseScope>();  
  }  
  
  @override  
  bool updateShouldNotify(ExpenseScope oldWidget) {  
    return expenses != oldWidget.expenses;  
  }  
}
```

Note: `Theme.of(context)` and `MediaQuery.of(context)` are powered by `InheritedWidget` !

Part 2: Practical State Management with Provider

Provider: The Pragmatic Wrapper

Provider wraps `InheritedWidget` in a clean, maintainable API:



- **Separation of Concerns:** Business logic lives in `ChangeNotifier`, UI stays declarative.
- **Selective Subtree Rebuilds:** Rebuilds only the nodes subscribed to changes.

Creating a **ChangeNotifier** Model

```
import 'package:flutter/foundation.dart';

class ExpenseNotifier extends ChangeNotifier {
  final List<ExpenseItem> _items = [];

  List<ExpenseItem> get items => List.unmodifiable(_items);
  double get grandTotal => _items.fold(0.0, (sum, i) => sum + i.amount);

  void addExpense(ExpenseItem item) {
    _items.add(item);
    notifyListeners(); // 🔔 Notifies active listening widgets to rebuild!
  }

  void removeExpense(String id) {
    _items.removeWhere((item) => item.id == id);
    notifyListeners();
  }
}
```

Providing & Consuming State

```
// 1. Mount provider at top of widget tree (main.dart)
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => ExpenseNotifier(),
      child: const MainApp(),
    ),
  );
}

// 2. Consume in UI using context.watch<T>()
class GrandTotalDisplay extends StatelessWidget {
  const GrandTotalDisplay({super.key});

  @override
  Widget build(BuildContext context) {
    final total = context.watch<ExpenseNotifier>().grandTotal;
    return Text("${total.toStringAsFixed(0)} đ");
  }
}
```

watch vs. read vs. select

Choosing the correct access pattern avoids expensive unintended rebuilds:

```
// 1. context.watch<T>()  
// ✓ Rebuilds whenever ANY notifyListeners() is called in the model  
final model = context.watch<ExpenseNotifier>();  
  
// 2. context.read<T>()  
// ✓ Reads once without subscribing. Use inside button callbacks!  
ElevatedButton(  
  onPressed: () => context.read<ExpenseNotifier>().addExpense(newItem),  
  child: const Text("Save"),  
);  
  
// 3. context.select<T, R>((model) => model.field)  
// ✓ Fine-grained subscription: Rebuilds ONLY when the selected value changes!  
final count = context.select<ExpenseNotifier, int>((m) => m.items.length);
```

Fine-Grained Optimization: **Consumer**

Instead of rebuilding an entire page's `build()` method, wrap only the label in **Consumer** :

```
class ExpenseDashboardPage extends StatelessWidget {
  const ExpenseDashboardPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        const ExpensiveStaticHeroImage(), // ⚡ NEVER rebuilds on state change
        const SizedBox(height: 16),
        Consumer<ExpenseNotifier>(
          builder: (context, notifier, child) {
            return Text("Current Balance: ${notifier.grandTotal} đ");
          },
        ),
      ],
    );
  }
}
```

Part 3: Modern Compile-Safe State with Riverpod 2

Why Riverpod 2?

Riverpod ("Provider" anagram) eliminates Provider's fundamental architectural flaws:

- **✗ No `ProviderNotFoundException`**: Providers are compile-time top-level constants — missing providers fail at **compile time**, not runtime!
- **✗ No `BuildContext` Dependency**: Read and manipulate state from services, background tasks, or testing suites without UI context.
- **✓ Built-in Async Safety**: Native `AsyncValue` handling for data, loading, and error states.
- **✓ Auto-Dispose**: Frees memory automatically when a screen closes: `ref.autoDispose`.
- **✓ Effortless Mocking**: Override any provider in widget tests with zero boilerplate.

```
flutter pub add flutter_riverpod
```

Defining a Riverpod 2 **Notifier**

```
// Notifier managing state mutations
class ExpenseController extends Notifier<List<ExpenseItem>> {
  @override
  List<ExpenseItem> build() => const []; // Initial state

  void add(ExpenseItem item) {
    state = [...state, item]; // Immutable update
  }
}

// Global compile-time safe provider definition
final expenseProvider = NotifierProvider<ExpenseController, List<ExpenseItem>>(
  ExpenseController.new,
);
```

Consuming Riverpod: ConsumerWidget

Replace `StatelessWidget` with `ConsumerWidget` to gain access to `WidgetRef` :

```
class ExpenseListView extends ConsumerWidget {
  const ExpenseListView({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final expenses = ref.watch(expenseProvider); // Reactive subscription

    return Scaffold(
      appBar: AppBar(title: Text("Expenses (${expenses.length})")),
      body: ListView.builder(
        itemCount: expenses.length,
        itemBuilder: (ctx, i) => ListTile(title: Text(expenses[i].title)),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => ref.read(expenseProvider.notifier).add(newItem),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```


Handling Async Data: FutureProvider

Riverpod turns complex asynchronous data streams into declarative UI states:

```
final receiptListProvider = FutureProvider<List<ExpenseItem>>((ref) async {
  return ExpenseDatabase.instance.getAllExpenses();
});

class AsyncReceiptList extends ConsumerWidget {
  const AsyncReceiptList({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return ref.watch(receiptListProvider).when(
      data: (items) => ListView.builder(
        itemCount: items.length,
        itemBuilder: (ctx, i) => ListTile(title: Text(items[i].title)),
      ),
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (err, _) => Center(child: Text("Error: $err")),
    );
  }
}
```

Flutter State Management Comparison Matrix

Solution	Learning Curve	Boilerplate	Compile Safety	Best Use Case
<code>setState()</code>	★ Low	None	✓ Safe	Ephemeral single-widget UI states
<code>InheritedWidget</code>	★★★★ High	High	✓ Safe	Library authors, low-level framework
<code>Provider</code>	★★ Medium	Low	⚠ Runtime lookup	Small-to-medium enterprise apps
<code>Riverpod 2</code>	★★ Medium	Very Low	✓ 100% Compile Safe	Modern multi-platform production apps
<code>BLoC</code>	★★★★ High	High	✓ Safe	Strict event-driven banking/fintech apps

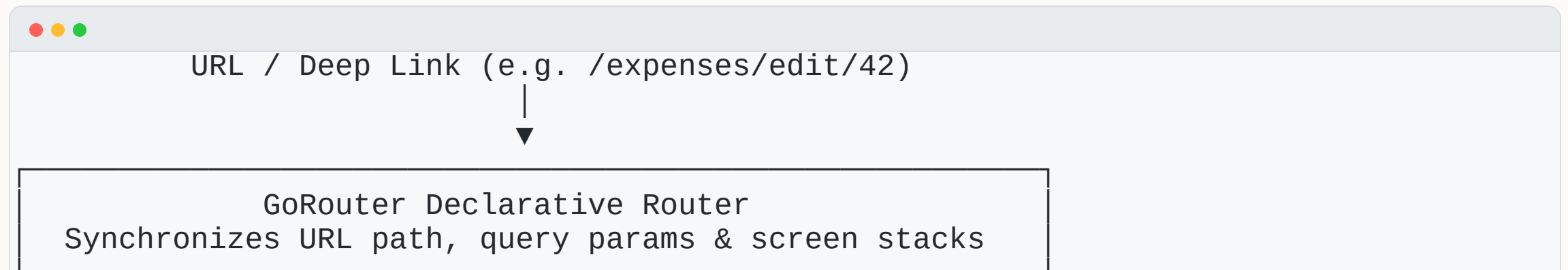
Recommendation: For Mini-Project 3 and modern Flutter development, **Riverpod 2** delivers the cleanest code, fastest testing workflow, and zero runtime provider crashes.

Part 4: Declarative Routing with GoRouter

Why Declarative Routing? (Navigator 1.0 vs. 2.0)

Traditional `Navigator.push(context, MaterialPageRoute(...))` fails on modern multi-platform apps:

- **✗ Deep Linking Failure:** Cannot handle external URLs like `myapp://receipts/scan?category=Food`.
- **✗ Browser Sync:** Web address bar does not update on navigation; browser Back button breaks.
- **✗ State Synchronization:** Screen stacks are mutated imperatively without a single source of truth.



Configuring GoRouter

```
import 'package:go_router/go_router.dart';

final appRouter = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(path: '/', builder: (context, state) => const HomeScreen()),
    GoRoute(path: '/scan', builder: (context, state) => const OCRScannerScreen()),
    GoRoute(
      path: '/expense/:id',
      builder: (context, state) {
        final id = state.pathParameters['id']!;
        return ExpenseDetailScreen(id: id);
      },
    ),
  ],
);

// Connect to MaterialApp in main.dart
MaterialApp.router(routerConfig: appRouter);
```

Navigating & Passing Parameters

```
// 1. Basic Navigation
context.go('/scan');    // Replaces stack with /scan
context.push('/scan'); // Pushes /scan on top of current stack

// 2. Dynamic Path Parameter
final String selectedId = "rec_9823";
context.push('/expense/$selectedId');

// 3. Query Parameters (/expenses?category=food)
context.push(
    Uri(path: '/expenses', queryParameters: {'category': 'food'}).toString(),
);

// Read query parameter inside builder:
final category = state.uri.queryParameters['category'] ?? 'All';

// 4. Return to previous screen
context.pop();
```

Persistent Tab Shell: `ShellRoute`

`ShellRoute` embeds nested screens inside a persistent parent layout:

```
ShellRoute(  
  builder: (context, state, child) => Scaffold(  
    body: child, // Nested active tab screen  
    bottomNavigationBar: NavigationBar(  
      selectedIndex: state.matchedLocation == '/scanner' ? 1 : 0,  
      onDestinationSelected: (idx) => context.go(idx == 0 ? '/dash' : '/scanner'),  
      destinations: const [  
        NavigationDestination(icon: Icon(Icons.home), label: 'Home'),  
        NavigationDestination(icon: Icon(Icons.camera), label: 'Scan'),  
      ],  
    ),  
  ),  
  routes: [  
    GoRoute(path: '/dash', builder: (ctx, s) => const DashboardTab()),  
    GoRoute(path: '/scanner', builder: (ctx, s) => const ScannerTab()),  
  ],  
);
```

Route Guards & Redirects

Protect private screens or redirect unverified users automatically:

```
final appRouter = GoRouter(  
  initialLocation: '/',  
  redirect: (BuildContext context, GoRouterState state) {  
    final bool isLoggedIn = AuthService.instance.isAuthenticated;  
    final bool isLoggingIn = state.matchedLocation == '/login';  
  
    // 1. If not logged in and not on login page, force redirect to /login  
    if (!isLoggedIn && !isLoggingIn) return '/login';  
  
    // 2. If logged in and attempting to access /login, redirect to home  
    if (isLoggedIn && isLoggingIn) return '/';  
  
    return null; // Return null means allow navigation  
  },  
  routes: [ /* ... */ ],  
);
```


Part 5: Form Handling, Validation & User Input

Form Architecture in Flutter

Mobile forms require strict validation, especially when auditing OCR extracted receipt amounts:

```
class EditReceiptForm extends StatefulWidget {  
  final ParsedReceipt initialData;  
  const EditReceiptForm({super.key, required this.initialData});  
  
  @override  
  State<EditReceiptForm> createState() => _EditReceiptFormState();  
}  
  
class _EditReceiptFormState extends State<EditReceiptForm> {  
  final _formKey = GlobalKey<FormState>();  
  late final _merchantController = TextEditingController(text: widget.initialData.merchantName);  
  late final _amountController = TextEditingController(text: "${widget.initialData.totalAmount ?? ''}");  
}
```

TextFormField & Custom Validation

```
Form(  
  key: _formKey,  
  autovalidateMode: AutovalidateMode.onUserInteraction,  
  child: Column(  
    children: [  
      TextFormField(  
        controller: _merchantController,  
        decoration: const InputDecoration(labelText: "Merchant Name"),  
        validator: (v) => (v == null || v.trim().isEmpty) ? "Required field" : null,  
      ),  
      TextFormField(  
        controller: _amountController,  
        keyboardType: TextInputType.number,  
        decoration: const InputDecoration(labelText: "Total Amount (VND)"),  
        validator: (v) {  
          final n = double.tryParse(v?.replaceAll(',', ''), ?? '');  
          return (n == null || n <= 0) ? "Enter a valid positive amount" : null;  
        },  
      ),  
    ],  
  ),  
)
```

Submitting & Focus Management

```
void _submitForm() {  
    if (_formKey.currentState!.validate()) {  
        _formKey.currentState!.save();  
  
        final merchant = _merchantController.text.trim();  
        final amount = double.parse(_amountController.text.replaceAll(',', ''));  
  
        ref.read(expenseProvider.notifier).add(  
            ExpenseItem(id: UniqueKey().toString(), title: merchant, amount: amount, timestamp: DateTime.now()),  
        );  
  
        ScaffoldMessenger.of(context).showSnackBar(  
            const SnackBar(content: Text("✅ Saved successfully!")),  
        );  
        context.pop();  
    }  
}
```

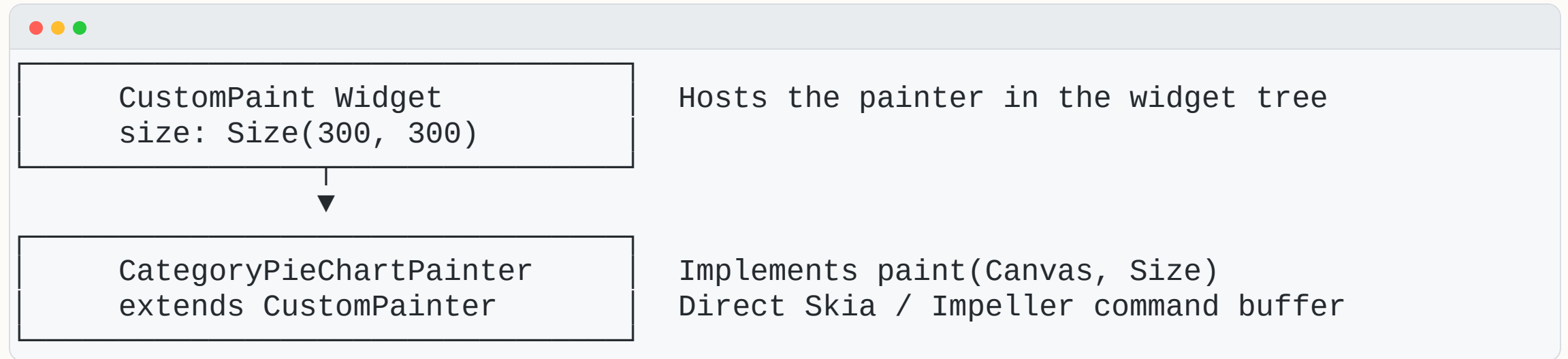
Use `FocusScope.of(context).requestFocus(_nextFocusNode)` to automatically advance focus on submit.

Part 6: Custom Canvas Drawing with **CustomPainter** & Animations

Why CustomPainter?

Off-the-shelf chart libraries are often bloated, lack custom styling, or fail 120Hz smooth animations.

Flutter's **CustomPainter** gives direct, hardware-accelerated access to the 2D graphics canvas:



- **`paint(Canvas canvas, Size size)`** : Issue raw drawing commands (`drawRect` , `drawArc` , `drawPath`).
- **`shouldRepaint(oldDelegate)`** : Determines if canvas must redraw when widget inputs change.

The Canvas Coordinate System

Flutter's canvas origin `(0, 0)` begins at the **top-left** corner:



- `Offset(dx, dy)` : Points on the 2D Cartesian plane.
- `Size(width, height)` : Bounding box dimensions passed to your painter.
- **Angles**: Expressed in **radians** (0 at 3 o'clock, $\pi/2$ at 6 o'clock, π at 9 o'clock).

Basic CustomPainter Anatomy

```
class SimpleLinePainter extends CustomPainter {  
  @override  
  void paint(Canvas canvas, Size size) {  
    final paint = Paint()  
      ..color = Colors.blue  
      ..strokeWidth = 3.0;  
    canvas.drawLine(Offset.zero, Offset(size.width, size.height), paint);  
  }  
  
  @override  
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;  
}
```

- **paint()** : Draws paths, rects, arcs, and text directly onto the GPU canvas.
- **shouldRepaint()** : Re-executes only when inputs change, avoiding re-draw overhead.

Animated Category Donut Chart: Painter

```
class DonutChartPainter extends CustomPainter {
    final List<double> percentages; // [0.4, 0.35, 0.25]
    final List<Color> colors;
    final double progress; // 0.0 -> 1.0

    DonutChartPainter({required this.percentages, required this.colors, required this.progress});

    @override
    void paint(Canvas canvas, Size size) {
        final center = Offset(size.width / 2, size.height / 2);
        final paint = Paint()..style = PaintingStyle.stroke..strokeWidth = 24.0;
        double startAngle = -pi / 2;

        for (int i = 0; i < percentages.length; i++) {
            final sweep = (2 * pi * percentages[i]) * progress;
            paint.color = colors[i % colors.length];
            canvas.drawArc(Rect.fromCircle(center: center, radius: 80), startAngle, sweep, false, paint);
            startAngle += sweep;
        }
    }

    @override
    bool shouldRepaint(covariant DonutChartPainter old) => old.progress != progress;
}
```

Wiring with `AnimationController`

Connect your painter with `AnimatedBuilder` to animate the sweep angle:

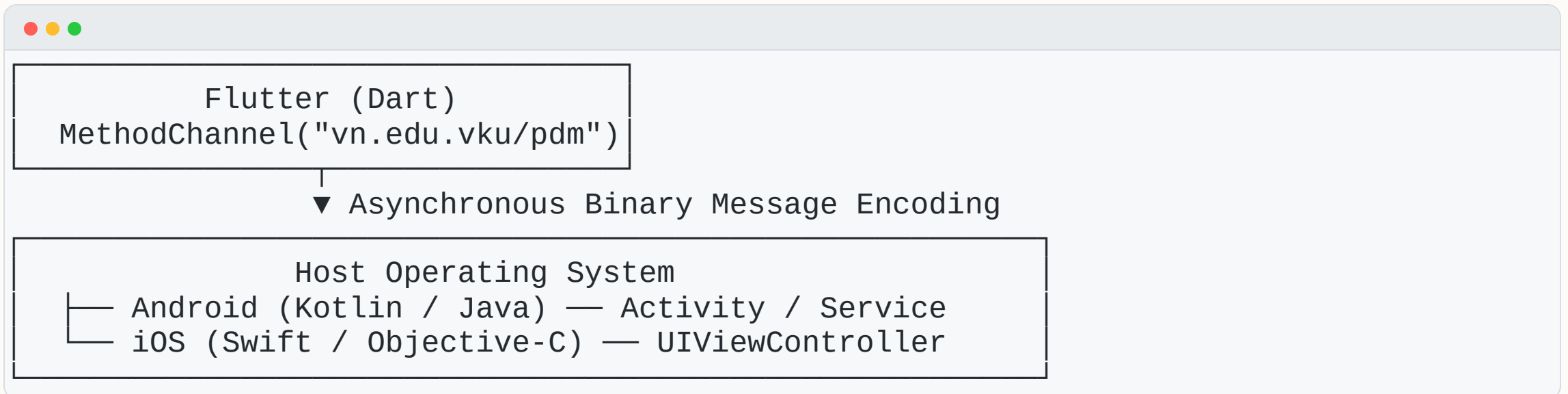
```
class AnimatedChart extends StatelessWidget {  
  final Animation<double> animation;  
  final List<double> data;  
  const AnimatedChart({super.key, required this.animation, required this.data});  
  
  @override  
  Widget build(BuildContext context) {  
    return AnimatedBuilder(  
      animation: animation,  
      builder: (context, _) => CustomPaint(  
        size: const Size(200, 200),  
        painter: DonutChartPainter(  
          percentages: data,  
          colors: const [Colors.orange, Colors.blue, Colors.purple],  
          progress: animation.value,  
        ),  
      ),  
    );  
  }  
}
```

Part 7: Platform Channels & Native Interoperability

When Pure Dart Isn't Enough

While Flutter plugins cover 95% of use-cases, engineers often need custom native bridges for:

- Hardware peripherals (thermal receipt printers, barcode lasers, biometrics).
- Proprietary enterprise SDKs (banking cryptographic tokens, legacy C++ libraries).
- Deep platform OS APIs (battery optimization whitelisting, background services).



The 3 Channel Flavors

Channel Type	Communication Pattern	Typical Real-World Use Case
MethodChannel	Request - Response (Async RPC)	Read battery %, trigger vibration haptics
EventChannel	Stream (Native → Dart events)	Continuous sensor stream (gyroscope, GPS)
BasicMessageChannel	Raw message passing	Custom protocol buffers, continuous image frames

```
// Dart Side: MethodChannel declaration
static const platform = MethodChannel('vn.edu.vku/device_info');

Future<int> getBatteryLevel() async {
  try {
    return await platform.invokeMethod('getBatteryLevel');
  } on PlatformException catch (e) {
    return -1;
  }
}
```

Native Implementation: Android (Kotlin)

Register the handler inside `MainActivity.kt` :

```
class MainActivity: FlutterActivity() {
    private val CHANNEL = "vn.edu.vku/device_info"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL)
            .setMethodCallHandler { call, result ->
                if (call.method == "getBatteryLevel") {
                    val bm = getSystemService(Context.BATTERY_SERVICE) as BatteryManager
                    result.success(bm.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPACITY))
                } else {
                    result.notImplemented()
                }
            }
    }
}
```

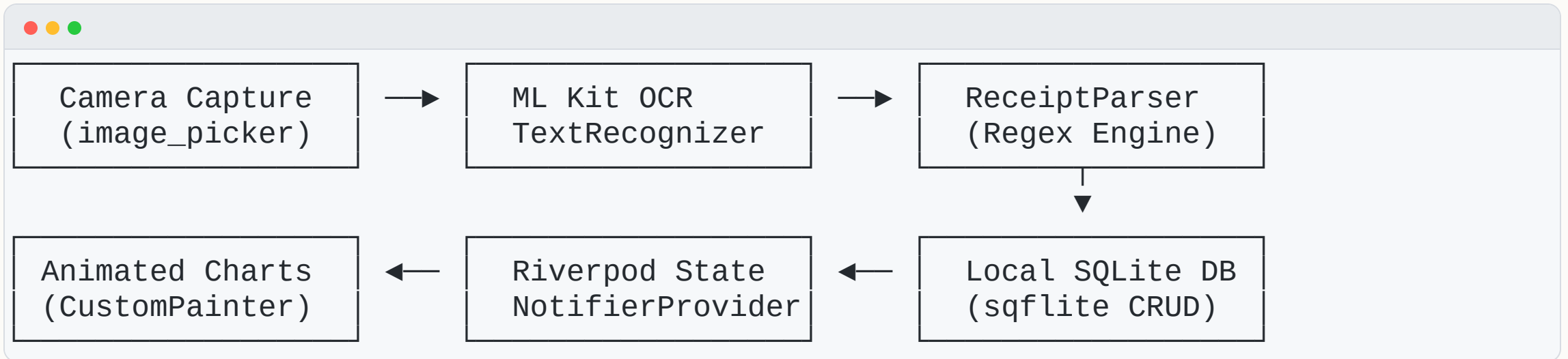
Native Implementation: iOS (Swift)

Register the handler inside `AppDelegate.swift` :

```
@objc class AppDelegate: FlutterAppDelegate {  
    override func application(  
        _ app: UIApplication, didFinishLaunchingWithOptions opts: [UIApplication.LaunchOptionsKey: Any]?  
    ) -> Bool {  
        let controller = window?.rootViewController as! FlutterViewController  
        let channel = FlutterMethodChannel(name: "vn.edu.vku/device_info", binaryMessenger: controller.binaryMessenger)  
  
        channel.setMethodCallHandler { (call, result) in  
            guard call.method == "getBatteryLevel" else { result(FlutterMethodNotImplemented); return }  
            UIDevice.current.isBatteryMonitoringEnabled = true  
            result(Int(UIDevice.current.batteryLevel * 100))  
        }  
        return super.application(app, didFinishLaunchingWithOptions: opts)  
    }  
}
```

Part 8: Mini-Project 3 Final Sprint

Mini-Project 3: Connecting the System Pipeline



Key Requirement: The OCR output is raw and noisy. Your app must feature a **Review & Verification Screen** where students can manually inspect the bounding values and correct OCR mistakes before committing records to SQLite.

Real-World Receipt Regex Tuning

Receipt formats in Vietnam are highly diverse (VNĐ , đ , commas vs dots):

```
class ReceiptParser {
    // Matches "TOTAL: 150.000", "Tổng tiền: 150,000 đ", "Thanh toán: 150000"
    static double? extractTotal(String rawText) {
        final keywordPattern = RegExp(r'(total|tong tien|thanh toan|amount due)', caseSensitive: false);
        final numberPattern = RegExp(r'[\d]{1,3}(?:[.,]\d{3})*(?:\.\d{2})?');

        for (final line in rawText.split('\n')) {
            if (keywordPattern.hasMatch(line)) {
                final matches = numberPattern.allMatches(line);
                if (matches.isNotEmpty) {
                    final raw = matches.last.group(0)!.replaceAll('.', ',').replaceAll(',', '.', '');
                    return double.tryParse(raw);
                }
            }
        }
        return null; // Fallback to manual entry if OCR missing
    }
}
```

Deliverables & Grading Checklist

- [] **1. Signed Release APK (`app-release.apk`)**: Downloadable from GitHub Releases or Google Drive. Tested on physical Android device.
- [] **2. 2–3 Minute Demo Video**: Video recording showing live camera scanning of a real receipt, instant regex parsing, review confirmation, and the animated donut chart updating.
- [] **3. GitHub Repository**: Public repository with clean architecture (`core/` , `models/` , `services/` , `state/` , `widgets/` , `screens/`), clean commits, and comprehensive `README.md` .
- [] **4. Short Technical Report (PDF)**: 2–4 pages adhering to the template: Architecture diagram, OCR regex table, screenshots of light/dark modes, and limitations.

Deadline: End of Week 8 (Sunday, 23:59). Late submissions lose 1 pt per day.

Next Week: Mobile Clean Architecture

In **Week 9**, we unify everything we've learned across PWA, Capacitor, React Native, and Flutter into **Enterprise Clean Architecture**:

- **Domain Layer:** Entities, Use Cases & Pure Business Rules (Zero UI or DB imports).
- **Data Layer:** Repository Pattern, Data Sources (Remote REST / Supabase + Local Cache).
- **Presentation Layer:** State Controllers & Dumb Widgets.
- **Automated Testing:** Unit testing use cases with mocks & automated Widget tests!